



IA CORPORATIVA COM JAVA E LANGCHAIN4J

AULA 04 - PORQUE RAG? ARQUITETURA DE UM
PIPELINE DE CONHECIMENTO



OBJETIVOS DA AULA

- ❑ Analisar as limitações críticas de um LLM: conhecimento estático (knowledge cutoff) e ausência de dados proprietários.
- ❑ Comparar as duas estratégias para fornecer conhecimento: RAG vs. Fine-Tuning, e entender por que RAG é a escolha arquitetural ideal para a maioria dos usos corporativos.
- ❑ Dissecar a arquitetura de um pipeline RAG, detalhando o propósito técnico de cada etapa nas fases de Ingestão e Recuperação.
- ❑ Entender como os conceitos de Janela de Contexto e Embeddings são aplicados na prática dentro de um pipeline RAG.

O DILEMA DO CONHECIMENTO DO LLM

O DILEMA DO CONHECIMENTO DO LLM

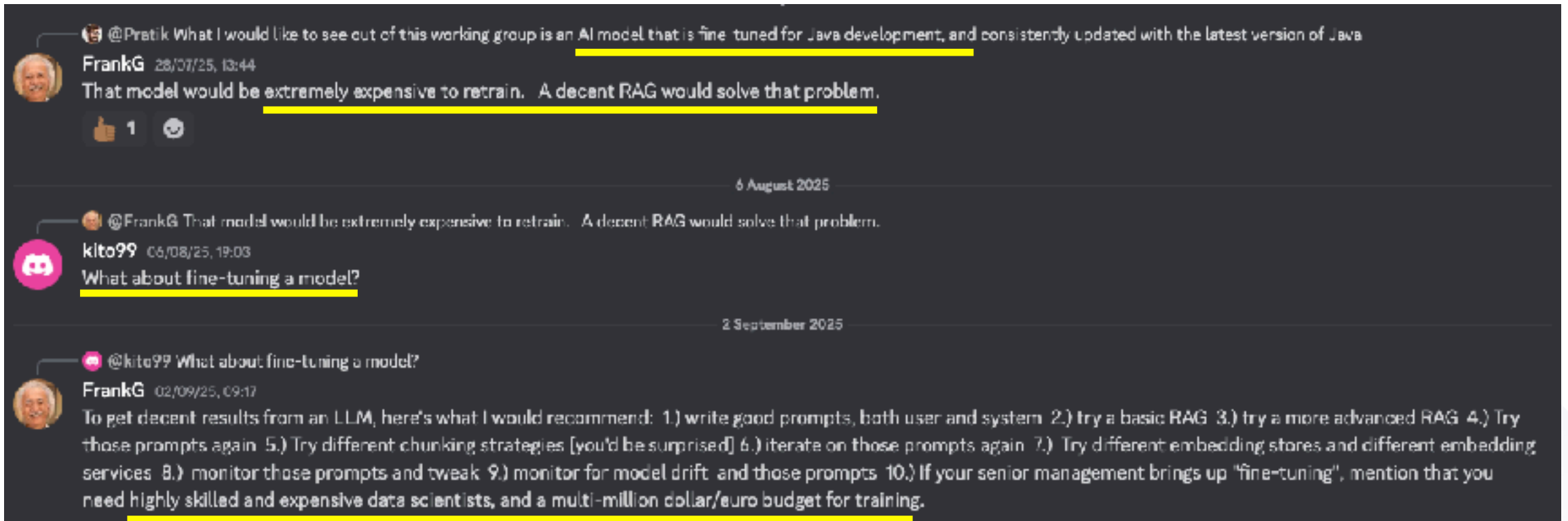
- ❑ O Ponto de Partida: Até então, nosso agente deu uma resposta genérica sobre perguntas genéricas, mas respondeu com precisão após ativarmos o Easy RAG e adicionarmos dados específicos.
- ❑ Isso ilustra o dilema fundamental dos LLMs pré-treinados :
 - ❑ Knowledge Cutoff (Conhecimento Estático): Um LLM é um artefato de software cujo conhecimento é congelado com os dados do seu treinamento. Ele não tem conhecimento de eventos, produtos ou dados gerados após esse evento.
 - ❑ The Walled Garden (Jardim Murado): O modelo não tem acesso a nenhuma informação que não seja pública e parte de seu massivo, porém genérico, dataset de treinamento. Ele é completamente isolado dos dados privados e proprietários da sua empresa (documentos internos, bases de dados, e-mails, etc.).

ANÁLISE COMPARATIVA: RAG VS. FINE-TUNING

- Existem duas estratégias principais para especializar um LLM: Fine-Tuning e RAG. Para um desenvolvedor corporativo, entender a diferença é uma decisão de arquitetura crucial.

Característica	Fine-Tuning (Re-treinamento)	RAG (Recuperação e Geração)
Objetivo Principal	Ensinar ao modelo um novo comportamento, estilo ou habilidade.	Fornecer ao modelo fatos e conhecimento para consulta.
Custo e Complexidade	Alto. Requer datasets massivos e curados, além de poder computacional significativo (GPUs) por longos períodos.	Menor. O custo está na engenharia de dados (pipeline de ingestão) e no armazenamento vetorial, não no re-treinamento do modelo.
Atualização	Lento e caro. Qualquer novo conhecimento exige um novo ciclo de fine-tuning.	Rápido e barato. Basta atualizar o documento na base de conhecimento e re-indexá-lo no Vector Database.
Risco de Alucinação	Ainda presente. O modelo pode "esquecer" fatos ou misturar o novo conhecimento com o antigo de forma imprevisível.	Reduzido. A resposta é "ancorada" nos fatos recuperados. O modelo é instruído a responder com base no contexto fornecido.
Rastreabilidade	Nenhuma. É impossível saber por que o modelo gerou uma resposta específica. É uma caixa-preta.	Alta. Podemos inspecionar quais trechos de quais documentos foram recuperados e usados para gerar a resposta.

ANÁLISE COMPARATIVA: RAG VS. FINE-TUNING



Fonte: Discussão que aconteceu recentemente no “*AI Java Working Group*”, grupo de trabalho estabelecido dentro dos Java Champions

ARQUITETURA DO PIPELINE

RAG: VISÃO GERAL

ARQUITETURA DO PIPELINE RAG: VISÃO GERAL

- ❑ Um pipeline RAG é melhor compreendido como dois processos distintos e desacoplados :
 1. Indexing Pipeline (Fase de Ingestão - Offline):
 - ❑ Propósito: Um processo de ETL (Extract, Transform, Load) que prepara os documentos da sua base de conhecimento e os indexa em um formato otimizado para busca semântica.
 - ❑ Quando Executar: Este é um processo offline. Ele roda na inicialização da aplicação, de forma agendada (ex: toda noite) ou acionado por eventos (ex: quando um novo documento é salvo no seu repositório).
 2. Retrieval and Generation Pipeline (Fase de Recuperação - Online/Runtime):
 - ❑ Propósito: O processo em tempo real que, dado uma pergunta do usuário, encontra a informação relevante e gera uma resposta.
 - ❑ Quando Executar: A cada requisição do usuário. A latência desta fase é crítica para a experiência do usuário.

FASE DE INGESTÃO (LOAD & SPLIT)

- ❑ Load (Carregar):
 - ❑ Este passo utiliza DocumentLoaders para carregar o conteúdo bruto de diversas fontes. Frameworks como o Langchain4j oferecem carregadores para arquivos (.txt, .pdf, .md), páginas web, repositórios Git, e mais. O objetivo é extrair o texto puro.
- ❑ Split (Dividir):
 - ❑ Este é um dos passos mais críticos. Não podemos simplesmente pegar um documento de 100 páginas e usá-lo. Usamos DocumentSplitters para quebrar o texto em chunks (segmentos) menores.
- ❑ Por que dividir?
 - ❑ Respeitar a Janela de Contexto: O LLM tem uma "memória RAM" limitada (a janela de contexto). Os chunks devem ser pequenos o suficiente para caberem, junto com o resto do prompt, dentro dessa janela.
 - ❑ Aumentar a Precisão da Busca: Uma busca semântica é mais eficaz em trechos focados. Se a pergunta é sobre "política de cancelamento", um chunk de 300 tokens focado nesse tópico é um resultado muito melhor do que um documento inteiro de 5000 tokens que menciona o termo apenas uma vez.
 - ❑ Otimizar Custo e Latência: Enviar menos tokens para o LLM em cada chamada resulta em menor latência e menor custo de API (quando aplicável).

FASE DE INGESTÃO (EMBED & STORE)

- ❑ Embed (Vetorizar):
 - ❑ Cada chunk de texto gerado no passo anterior é passado para um EmbeddingModel.
 - ❑ Este modelo converte o texto em um vetor numérico de alta dimensão (um embedding), que captura a sua essência semântica. É neste ponto que o "significado" do texto é traduzido para uma representação matemática que pode ser comparada.
- ❑ Store (Armazenar):
 - ❑ Os chunks de texto e seus respectivos embeddings são armazenados em um Vector Database (ou Vector Store).
 - ❑ Vector Database: É um banco de dados especializado, otimizado para armazenar e consultar vetores de alta dimensão. Sua principal operação não é um `SELECT... WHERE text LIKE '%termo%'`, mas sim uma busca por similaridade, que encontra os vetores mais "próximos" de um dado vetor de consulta de forma extremamente eficiente.

BUSCA VETORIAL (RETRIEVE)

- ❑ Desafio da Dimensionalidade
 - ❑ Uma busca exata (brute-force) que compara o vetor da pergunta com todos os vetores no banco de dados é computacionalmente inviável em escala. Este problema é conhecido como a "maldição da dimensionalidade".
- ❑ Solução
 - ❑ Approximate Nearest Neighbor (ANN):
 - ❑ Em vez de garantir o vizinho mais próximo, os sistemas de produção usam algoritmos ANN para encontrar vizinhos "próximos o suficiente" de forma extremamente rápida. Eles trocam uma pequena fração de precisão por uma enorme melhoria em velocidade.
 - ❑ HNSW (Hierarchical Navigable Small World)
 - ❑ É um dos algoritmos ANN mais populares e de melhor desempenho. Ele constrói um grafo de múltiplas camadas onde as camadas superiores têm links longos (para navegar rapidamente pelo espaço) e as camadas inferiores têm links curtos (para refinar a busca com precisão).
- ❑ Métrica vs. Algoritmo:
 - ❑ Similaridade de Cossenos: É a métrica matemática usada para calcular o quão "próximos" dois vetores estão, medindo o ângulo entre eles.
 - ❑ ANN (HNSW): É o algoritmo (o motor) que usa essa métrica para realizar a busca de forma eficiente em milhões de vetores sem ter que comparar todos eles.

OTIMIZANDO O PIPELINE DE RECUPERAÇÃO



OTIMIZANDO O PIPELINE DE RECUPERAÇÃO

- ❑ O Problema da Recuperação Inicial
 - ❑ A primeira fase de recuperação (via ANN) é otimizada para velocidade e recall (trazer muitos documentos potencialmente relevantes), mas não para precisão. Ela pode retornar documentos que são semanticamente próximos, mas não perfeitamente relevantes para a nuance da pergunta.
- ❑ Solução
 - ❑ Um Pipeline de Duas Etapas com Re-ranking.
 - ❑ Etapa 1
 - ❑ Recuperação Ampla (Bi-Encoder): Um recuperador rápido (como o que usa ANN) busca um conjunto maior de documentos candidatos (ex: top 20 a 50).
 - ❑ Etapa 2
 - ❑ Re-ranking Preciso (Cross-Encoder): Um segundo modelo, mais lento e sofisticado, chamado cross-encoder, é usado. Diferente do bi-encoder, que processa a pergunta e os documentos separadamente, o cross-encoder avalia a pergunta e cada documento juntos, permitindo uma análise de relevância muito mais profunda e contextual. Ele então reordena a lista de candidatos, colocando os mais relevantes no topo.
 - ❑ Benefício: Esta abordagem combina a velocidade da busca ANN com a precisão de um modelo de análise mais profundo, melhorando drasticamente a qualidade dos documentos que são finalmente enviados ao LLM.

O PROBLEMA "LOST IN THE MIDDLE"

- ❑ Janela de Contexto Não é Uniforme
 - ❑ Pesquisas demonstraram que os LLMs não prestam atenção igual a todas as partes do seu contexto. Eles tendem a lembrar melhor das informações localizadas no início e no final do prompt, um fenômeno conhecido como "*Lost in the Middle*" (Perdido no Meio).
- ❑ Implicação Arquitetural
 - ❑ A ordem em que colocamos os documentos recuperados e re-ordenados no prompt é crucial. Se o documento mais importante for colocado no meio de uma longa lista de outros documentos, há uma chance significativa de que o LLM o ignore ou não lhe dê a devida importância.
- ❑ Melhor Prática
 - ❑ Após o re-ranking, os documentos mais relevantes devem ser posicionados estrategicamente no prompt. Uma prática comum é colocar o documento mais importante no final do contexto (imediatamente antes da pergunta) e o segundo mais importante no início, com os demais preenchendo o meio. Isso maximiza a probabilidade de que a atenção do modelo se concentre nas informações mais críticas.

FLUXO DO PIPELINE RAG

Fase	Etapa	Descrição
Ingestão (Offline)	1. Load & Split	Documentos brutos são carregados e divididos em chunks menores.
	2. Embed & Store	Cada chunk é convertido em um embedding e armazenado em um Vector Database.
Recuperação (Online)	1. Embed Query	A pergunta do usuário é convertida em um embedding.
	2. Retrieve	Uma busca ANN (HNSW) rápida é feita no Vector DB para recuperar um conjunto amplo de candidatos (ex: Top 50).
	3. Re-rank	Um Cross-Encoder analisa a pergunta contra cada candidato e reordena a lista para máxima relevância.
	4. Augment	O prompt final é montado, posicionando os documentos mais relevantes no início e no fim para evitar o "Lost in the Middle".
	5. Generate	O prompt aumentado é enviado ao LLM para gerar a resposta final, agora ancorada em dados precisos e bem posicionados.

RECAP E PRÓXIMOS PASSOS

RECAP E PRÓXIMOS PASSOS

❑ Recap

- ❑ RAG é a solução arquitetural preferida para enriquecer LLMs com conhecimento proprietário, superando o Fine-Tuning em custo, agilidade e rastreabilidade.
- ❑ A busca vetorial em produção depende de algoritmos ANN como HNSW para ser eficiente, usando métricas como a similaridade de cossenos.
- ❑ Pipelines de RAG de alta qualidade frequentemente usam um processo de duas etapas com re-ranking (cross-encoders) para melhorar a precisão da recuperação.
- ❑ A ordem do contexto no prompt importa devido ao problema "*Lost in the Middle*", exigindo o posicionamento estratégico dos documentos mais relevantes.

❑ Próximos passos

- ❑ Revisar o que a extensão `quarkus-langchain4j-easy-rag` faz "*por baixo dos panos*".
- ❑ Aprofundar na configuração do pipeline de ingestão, focando nos parâmetros de divisão de documentos (splitting).
- ❑ Aprender a otimizar a segmentação de documentos com `max-segment-size` e `max-overlap-size`.
- ❑ Utilizar a Dev UI do Quarkus para inspecionar o `EmbeddingStore` em memória e validar a ingestão.
- ❑ Testar o `AiService` para verificar o uso do conhecimento ingerido e otimizado.

ATÉ A PRÓXIMA!